

Lab Ngày 26: Hạ Tầng MCP/A2A & Agentic Routing

Khóa học: AICB-P2T2 · Tuần 6 · Chương 6

Framework: [Google Agent Development Kit \(ADK\)](#)

Thời lượng: ~2 giờ

Câu hỏi mở đầu

"Agent gọi agent — hạ tầng của bạn có scale được cho multi-agent choreography không?"

Một hệ thống nghiên cứu cần 4 agent chuyên biệt phối hợp. Nếu không có chuẩn giao tiếp chung, mỗi agent nói "ngôn ngữ" riêng. **MCP + A2A** là giao thức phổ quát cho hệ sinh thái AI agent.

Mục tiêu học tập

Sau buổi lab này, bạn sẽ:

- Thiết kế và triển khai **hạ tầng MCP server** cho AI tools
- Triển khai **giao tiếp A2A** giữa nhiều agent bằng ADK
- Xây dựng **lớp agentic routing** với semantic routing
- Áp dụng các mẫu **state, bảo mật và observability** trong hệ multi-agent

Sản phẩm cuối ngày

Thành phần	Yêu cầu
MCP server	3 tools qua stdio (local) hoặc SSE (remote)
Agent registry	Health check + khám phá capability
Semantic router	Định tuyến request tới đúng specialist
Demo multi-agent	Orchestrator + 2 specialist qua A2A
Trace	Toàn bộ luồng multi-agent hiển thị trong log/trace
Data governance	Policy MCP/A2A + audit log + HITL gate

Module 0 — Cài đặt môi trường

Lab dùng **Conda** (môi trường khuyến nghị: `pii-env`). Trước khi chạy cell bên dưới:

```
conda create -n pii-env python=3.12 -y # chỉ lần đầu
conda activate pii-env
```

Chọn kernel **pii-env** trong Jupyter nếu notebook hỏi.

Cài ADK kèm **extra A2A** và MCP SDK. Gói `google-adk` cơ bản **không** bao gồm dependency A2A.

```
In [1]: # Chạy một lần sau khi conda activate pii-env (kernel Jupyter = pii-env)
!pip install -q -r requirements.txt

# Nếu vẫn thấy cảnh báo cryptography từ presidio cũ (2.2.360), nâng cấp:
# !pip install -q -U "presidio-anonymizer>=2.2.363"
```

```
ERROR: Could not install packages due to an OSError: [Errno 2] No such file or directory: 'C:\\Users\\Admin\\AppData\\Local\\Packages\\PythonSoftwareFoundation.Python.3.11_qbz5n2kfra8p0\\LocalCache\\local-packages\\Python311\\site-packages\\jupyterlab\\galata\\@jupyterlab\\galata-extension\\static\\vendors-node_modules_fontsource-variable_noto-sans-sc_index_css.386da7677f8b1f52.js'
```

```
HINT: This error might have occurred since this system does not have Windows Long Path support enabled. You can find information on how to enable this at https://pip.pypa.io/warnings/enable-long-paths
```

```
[notice] A new release of pip is available: 24.0 -> 26.1.2
```

```
[notice] To update, run: C:\\Users\\Admin\\AppData\\Local\\Microsoft\\WindowsApps\\PythonSoftwareFoundation.Python.3.11_qbz5n2kfra8p0\\python.exe -m pip install --upgrade pip
```

```
In [1]: import os
from pathlib import Path

from dotenv import load_dotenv

PROJECT_ROOT = Path.cwd()
load_dotenv(PROJECT_ROOT / ".env")

# Google AI Studio (khuyến nghị cho Lab)
os.environ.setdefault("GOOGLE_GENAI_USE_VERTEXAI", "FALSE")

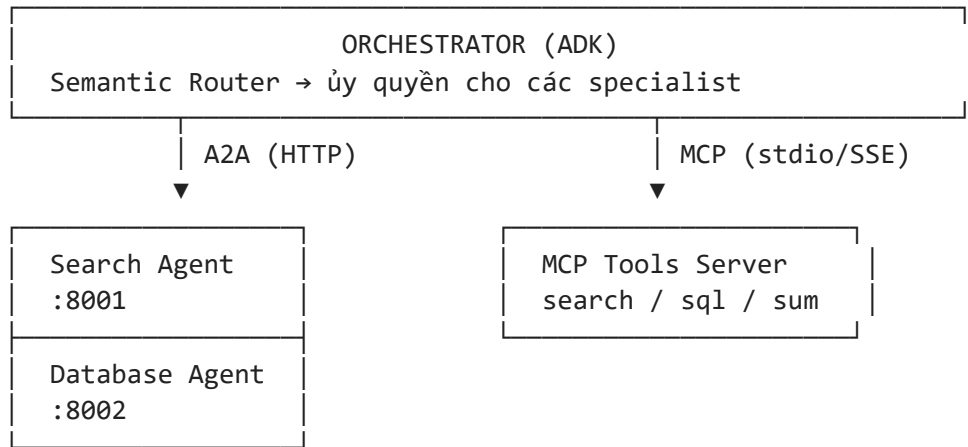
assert os.getenv("GOOGLE_API_KEY"), (
    "Đặt GOOGLE_API_KEY trong .env – lấy key tại https://aistudio.google.com/app/ap"
)

print("✓ Môi trường sẵn sàng")
print(f"Thư mục dự án: {PROJECT_ROOT}")
```

✓ Môi trường sẵn sàng

Thư mục dự án: D:\Learning\project\Day26-Track02-Cohort2-MCP-A2A-Infrastructure

Tổng quan kiến trúc



Điểm then chốt: MCP chuẩn hóa truy cập *tool*; A2A chuẩn hóa giao tiếp *agent*. ADK kết nối cả hai.

⚠ Bước bắt buộc — Khởi động A2A Specialist Servers

Phải làm trước Module 2, Module 4 và Capstone. Nếu bỏ qua, kiểm tra agent card sẽ báo [CHƯA ĐẠT] .

Điều kiện: đã cài dependency (Module 0) và **đang ở thư mục dự án**. Kích hoạt conda env:

```
conda activate pii-env
export PYTHONPATH=$PWD
```

Cách 1 — Từ notebook (khuyến nghị)

Chạy cell bên dưới để khởi động cả hai server nền. Đợi thông báo `search OK / database OK` , rồi chạy cell kiểm tra.

Cách 2 — Hai terminal riêng

```
cd Day26-MCP_A2A_Infrastructure
conda activate pii-env
export PYTHONPATH=$PWD
```

Terminal 1

```
python -m uvicorn agents.search_agent.agent:a2a_app --host localhost --port 8001
```

Terminal 2

```
python -m uvicorn agents.database_agent.agent:a2a_app --host localhost --port 8002
```

Hoặc: `bash scripts/start_search_agent.sh` và `bash scripts/start_database_agent.sh`

Cách 3 — Một lệnh (nền)

`bash scripts/start_a2a_servers.sh`

Kiểm tra / Dừng

```
curl http://localhost:8001/.well-known/agent-card.json
curl http://localhost:8002/.well-known/agent-card.json
bash scripts/stop_a2a_servers.sh
```

```
In [3]: # Khởi động A2A servers nền (chạy cell này một lần mỗi phiên Lab)
# Bản Windows-friendly: nếu server đã chạy sẵn thì dùng luôn, chưa thì spawn uvicorn
import os
import subprocess
import sys
import time

import httpx

PORTS = {"search_agent": 8001, "database_agent": 8002, "synthesis_agent": 8003}
os.makedirs(PROJECT_ROOT / "logs", exist_ok=True)

def _card_up(port: int) -> bool:
    try:
        url = f"http://localhost:{port}/.well-known/agent-card.json"
        return httpx.get(url, timeout=3).status_code == 200
    except Exception:
        return False

for name, port in PORTS.items():
    if _card_up(port):
        print(f"✓ {name} :{port} đã chạy sẵn")
        continue
    print(f"→ Khởi động {name} :{port} ...")
    log = open(PROJECT_ROOT / "logs" / f"{name}.log", "w")
    subprocess.Popen(
        [sys.executable, "-m", "uvicorn", f"agents.{name}.agent:a2a_app",
         "--host", "localhost", "--port", str(port)],
        cwd=PROJECT_ROOT, stdout=log, stderr=subprocess.STDOUT,
        env={**os.environ, "PYTHONPATH": str(PROJECT_ROOT)},
    )

for _ in range(15):
    if all(_card_up(p) for p in PORTS.values()):
        break
    time.sleep(2)

print()
```

```
for name, port in PORTS.items():
    print(("✓" if _card_up(port) else "X"), f"{name} :{port}")
```

✓ search_agent :8001 đã chạy sẵn
 ✓ database_agent :8002 đã chạy sẵn
 ✓ synthesis_agent :8003 đã chạy sẵn
 ✓ search_agent :8001
 ✓ database_agent :8002
 ✓ synthesis_agent :8003

```
In [4]: # Kiểm tra agent card – phải thấy ✓ trước khi sang Module 4 / Capstone
import httpx

AGENT_CARDS = {
    "search_agent": "http://localhost:8001/.well-known/agent-card.json",
    "database_agent": "http://localhost:8002/.well-known/agent-card.json",
    "synthesis_agent": "http://localhost:8003/.well-known/agent-card.json",
}

all_ok = True
for name, url in AGENT_CARDS.items():
    try:
        r = httpx.get(url, timeout=5)
        r.raise_for_status()
        card = r.json()
        print(f"✓ {name}: {card.get('name')} – {str(card.get('description', ''))[:50]}")
    except Exception as exc:
        all_ok = False
        print(f"X {name}: {exc}")
        port = {"search_agent": 8001, "database_agent": 8002, "synthesis_agent": 8003}
        print(f" → Chạy: uvicorn agents.{name}.agent:a2a_app --host localhost --port {port[name]}")

if all_ok:
    print("\n✓ A2A servers sẵn sàng – tiếp tục lab")
else:
    print("\nX Chưa sẵn sàng – chạy: bash scripts/start_a2a_servers.sh")
```

✓ search_agent: search_agent – Tìm kiếm web và trả về đoạn trích liên quan cho t
 á...
 ✓ database_agent: database_agent – Truy vấn database chỉ đọc và trả về metrics có c
 ấu...
 ✓ synthesis_agent: synthesis_agent – Tổng hợp kết quả nghiên cứu thành báo cáo cuối
 có ...
 ✓ A2A servers sẵn sàng – tiếp tục lab

Module 1 — Model Context Protocol (MCP)

Lý thuyết (từ slide)

Lớp	Vai trò
Tools Server	Thực thi hàm (search, SQL, API)

Lớp	Vai trò
Resources Server	Expose dữ liệu (file, hàng DB)
Prompts Server	Template prompt tái sử dụng

Transport: `stdio` (dev local), `HTTP+SSE` (remote/K8s), `WebSocket` (streaming hai chiều)

Quy tắc thiết kế:

- Docstring rõ ràng (LLM đọc để chọn tool)
- Type hint cho mọi tham số
- Validate input trước khi thực thi
- SQL tool mặc định chỉ đọc (read-only)
- Log mọi lần gọi tool

Bài tập 1.1 — Khám phá MCP Server

Mở `mcp_server/research_tools_server.py` và trả lời:

1. Ba tool nào được expose?
2. `_sql_query` enforce governance như thế nào?
3. Vì sao dùng transport `stdio` khi dev local?

```
In [5]: # Demo: gọi Logic MCP tool trực tiếp (không cần khởi động server)
import sys
sys.path.insert(0, str(PROJECT_ROOT / "mcp_server"))

from research_tools_server import _search_documents, _sql_query, _summarize_text

print("=== search_documents ===")
print(_search_documents("MCP"))

print("\n=== sql_query (cho phép) ===")
print(_sql_query("SELECT * FROM agent_metrics"))

print("\n=== sql_query (bị chặn) ===")
try:
    _sql_query("DROP TABLE agent_metrics")
except ValueError as exc:
    print(f"Đã chặn: {exc}")

print("\n=== summarize_text ===")
print("\n".join(_summarize_text("MCP build một lần. A2A kết nối agent. Routing chọn
```

```

=== search_documents ===
[{'id': 'doc-1', 'title': 'VinUni AI Curriculum Overview', 'body': 'Phase 2 Track 2 covers MCP, A2A, and multi-agent orchestration.'}, {'id': 'doc-2', 'title': 'MCP Transport Options', 'body': 'MCP supports stdio for local dev and HTTP+SSE for remote deployment.'}]

=== sql_query (cho phép) ===
[{'agent': 'search_agent', 'tasks_completed': 42, 'avg_latency_ms': 820}, {'agent': 'database_agent', 'tasks_completed': 31, 'avg_latency_ms': 1100}, {'agent': 'synthesis_agent', 'tasks_completed': 28, 'avg_latency_ms': 2400}]

=== sql_query (bị chặn) ===

=== summarize_text ===
- MCP build một lần
- A2A kết nối agent
- Routing chọn specialist

```

Kết nối MCP Tools với ADK Agent

ADK bọc MCP server qua `McpToolset`. Agent orchestrator trong repo đã bind cả 3 tool.

```

In [6]: from google.adk.tools.mcp_tool import StdioConnectionParams
        from google.adk.tools.mcp_tool.mcp_toolset import McpToolset
        from mcp import StdioServerParameters
        import sys

        mcp_server_path = PROJECT_ROOT / "mcp_server" / "research_tools_server.py"

        mcp_toolset = McpToolset(
            connection_params=StdioConnectionParams(
                server_params=StdioServerParameters(
                    command=sys.executable, # đúng Python của env Lab
                    args=[str(mcp_server_path)],
                ),
                timeout=10,
            ),
            tool_filter=["search_documents", "sql_query", "summarize_text"],
            # errlog: file thật - sys.stderr trong Jupyter không có fileno()
            errlog=open(PROJECT_ROOT / "logs" / "mcp_stderr.log", "a", encoding="utf-8"),
        )

        print("✓ Đã cấu hình McpToolset")
        print(f"  Server: {mcp_server_path.name}")
        print(f"  Bộ lọc: search_documents, sql_query, summarize_text")

```

```

✓ Đã cấu hình McpToolset
  Server: research_tools_server.py
  Bộ lọc: search_documents, sql_query, summarize_text

```

```

D:\envs\pii-env\Lib\site-packages\google\adk\features\_feature_decorator.py:72: User
Warning: [EXPERIMENTAL] feature FeatureName.PLUGGABLE_AUTH is enabled.
  check_feature_enabled()

```



Bài tập 1.2 — Thêm tool MCP thứ tư

Nhiệm vụ: Mở rộng `research_tools_server.py` với tool `count_words` trả về số từ trong chuỗi.

Checklist:

- ☐ Thêm vào `list_tools()` với `inputSchema` đúng
- ☐ Triển khai trong `call_tool()`
- ☐ Thêm vào `tool_filter` trong orchestrator agent
- ☐ Test qua ADK agent hoặc gọi hàm trực tiếp

Module 2 — Giao thức Agent-to-Agent (A2A)

Lý thuyết

A2A coi agent như **microservices**:

Khái niệm	Mô tả
Agent Card	JSON tại <code>/.well-known/agent-card.json</code> — capability & schema
Task	Đơn vị công việc với các trạng thái vòng đời
Message	Giao tiếp trong một task

Vòng đời Task

Submitted → Working → Input Required → Completed
 ↳ Failed / Canceled

Task có thể **tạm dừng** ở `input-required` — agent yêu cầu caller cung cấp thêm thông tin trước khi tiếp tục.

Ánh xạ sang ADK

Hành động	API ADK
Expose agent làm A2A server	<code>to_a2a(agent, port=8001)</code>
Tiêu thụ agent remote	<code>RemoteA2aAgent(name, description, agent_card=url)</code>
Thay thế qua CLI	<code>adk api_server --a2a --port 8001 agents/search_agent</code>

 **Thực hành — Kiểm tra lại A2A (Module 2)**

Đã khởi động ở Module 0.5 chưa? Nếu chưa, quay lại phần **[Bước bắt buộc — Khởi động A2A Specialist Servers]** và chạy cell khởi động + kiểm tra.

Cell bên dưới lặp lại bước xác nhận agent card (giống Module 0.5).

```
In [7]: import httpx

AGENT_CARDS = {
    "search": "http://localhost:8001/.well-known/agent-card.json",
    "database": "http://localhost:8002/.well-known/agent-card.json",
}

for label, url in AGENT_CARDS.items():
    try:
        response = httpx.get(url, timeout=3)
        response.raise_for_status()
        card = response.json()
        print(f"✓ {label}_agent: {card.get('name')} – {card.get('description', '')}")
    except Exception as exc:
        print(f"✗ Không kết nối được {label}_agent: {exc}")
        print(f"  Khởi động: uvicorn agents.{label}_agent.agent:a2a_app --port {8001 if label == 'search' else 8002}")
```

✓ search_agent: search_agent – Tìm kiếm web và trả về đoạn trích liên quan cho tác vụ nghiê...
✓ database_agent: database_agent – Truy vấn database chỉ đọc và trả về metrics có cấu trúc....

Tiêu thụ Remote Agent từ Orchestrator

Orchestrator dùng RemoteA2aAgent làm sub-agent. LLM coordinator của ADK ủy quyền dựa trên description của từng sub-agent.

```
In [8]: from google.adk.agents.remote_a2a_agent import RemoteA2aAgent

remote_search = RemoteA2aAgent(
    name="search_agent",
    description="Tìm kiếm tài liệu và web để thu thập bằng chứng nghiên cứu.",
    agent_card=AGENT_CARDS["search"],
)

remote_database = RemoteA2aAgent(
    name="database_agent",
    description="Chạy SQL chỉ đọc trên bảng metrics của agent.",
    agent_card=AGENT_CARDS["database"],
)

print("✓ Đã cấu hình proxy RemoteA2aAgent")
print(f"  search → {AGENT_CARDS['search']}")
print(f"  database → {AGENT_CARDS['database']}")
```

✓ Đã cấu hình proxy RemoteA2aAgent
search → http://localhost:8001/.well-known/agent-card.json
database → http://localhost:8002/.well-known/agent-card.json

```
C:\Users\Admin\AppData\Local\Temp\ipykernel_24932\3397977623.py:3: UserWarning: [EXPERIMENTAL] RemoteA2aAgent: ADK Implementation for A2A support (A2aAgentExecutor, RemoteA2aAgent and corresponding supporting components etc.) is in experimental mode and is subject to breaking changes. A2A protocol and SDK are themselves not experimental. Once it's stable enough the experimental mode will be removed. Your feedback is welcome.
    remote_search = RemoteA2aAgent(
C:\Users\Admin\AppData\Local\Temp\ipykernel_24932\3397977623.py:9: UserWarning: [EXPERIMENTAL] RemoteA2aAgent: ADK Implementation for A2A support (A2aAgentExecutor, RemoteA2aAgent and corresponding supporting components etc.) is in experimental mode and is subject to breaking changes. A2A protocol and SDK are themselves not experimental. Once it's stable enough the experimental mode will be removed. Your feedback is welcome.
    remote_database = RemoteA2aAgent(
```

 Bài tập 2.1 — A2A vs Sub-Agent Local

Điền bảng theo hiểu biết của bạn:

Tiêu chí	A2A (Remote)	Sub-Agent Local
Triển khai	?	?
Hiệu năng	?	?
Cô lập state	?	?
Phù hợp khi	?	?

Thảo luận: Khi nào chọn A2A thay vì `sub_agents=[local_agent]` trong ADK?

Module 3 — Mẫu Agentic Routing

So sánh chiến lược routing

Chiến lược	Ưu điểm	Nhược điểm	Khi nào dùng
Keyword	Nhanh, đơn giản	Dễ vỡ, bỏ sót ngữ cảnh	≤5 agent
Embedding	Bền, scale tốt	Chậm hơn, cần embedding	5–50 agent
LLM-based	Linh hoạt, hiểu ngữ cảnh	Đắt, chậm	Routing phức tạp

Semantic routing: embed request → cosine similarity với mô tả capability của agent → top-k ứng viên.

Fallback chain: agent chính → agent dự phòng → leo thang lên người — không để user bị kẹt.

```
In [9]: import sys
sys.path.insert(0, str(PROJECT_ROOT))

from lab_utils.semantic_router import AgentCapability, SemanticRouter

router = SemanticRouter(
    agents=[
        AgentCapability(
            name="search_agent",
            description="Tìm kiếm web tài liệu nghiên cứu bằng chứng",
            tags=["search", "web", "documents", "tìm kiếm", "bài viết", "tra cứu"],
        ),
        AgentCapability(
            name="database_agent",
            description="SQL metrics phân tích database truy vấn SELECT",
            tags=["sql", "metrics", "database"],
        ),
        AgentCapability(
            name="synthesis_agent",
            description="Tóm tắt kết hợp kết quả thành báo cáo cuối",
            tags=["summary", "report", "synthesis"],
        ),
    ],
)

test_queries = [
    "Tìm bài viết về việc áp dụng giao thức MCP",
    "SELECT độ trễ trung bình từ agent_metrics",
    "Viết tóm tắt một trang về kết quả nghiên cứu",
    "Xin chào, bạn làm được gì?",
]

print(f"{'Truy vấn':<50} {'Định tuyến tới':<20} Điểm")
print("-" * 80)
for query in test_queries:
    candidates = router.route(query, top_k=1)
    target = router.route_with_fallback(query, fallback="orchestrator")
    score = candidates[0][1] if candidates else 0.0
    print(f"{query[:48]:<50} {target:<20} {score:.3f}")
```

Truy vấn	Định tuyến tới	Điểm
Tìm bài viết về việc áp dụng giao thức MCP	search_agent	0.248
SELECT độ trễ trung bình từ agent_metrics	database_agent	0.257
Viết tóm tắt một trang về kết quả nghiên cứu	synthesis_agent	0.408
Xin chào, bạn làm được gì?	orchestrator	0.000

Mẫu Agent Registry

Trên production, agent đăng ký khi khởi động và hủy đăng ký khi tắt. Registry lưu metadata capability để discovery.

```
In [10]: from lab_utils.agent_registry import AgentRegistry, RegisteredAgent
```

```

registry = AgentRegistry()

registry.register(
    RegisteredAgent(
        name="search_agent",
        url="http://localhost:8001",
        description="Chuyên gia tìm kiếm web và tài liệu",
        capabilities={"skills": ["search_web"], "transport": "a2a"},
    )
)

registry.register(
    RegisteredAgent(
        name="database_agent",
        url="http://localhost:8002",
        description="Chuyên gia SQL metrics chỉ đọc",
        capabilities={"skills": ["run_sql_query"], "transport": "a2a"},
    )
)

print("Các agent đã đăng ký:")
for agent in registry.list_agents():
    status = "khỏe mạnh" if agent.healthy else "không khỏe"
    print(f" • {agent.name} ({status}) @ {agent.url}")

print("\nTìm capability 'sql':")
for agent in registry.find_by_capability("sql"):
    print(f" → {agent.name}")

```

Các agent đã đăng ký:

- search_agent (khỏe mạnh) @ http://localhost:8001
- database_agent (khỏe mạnh) @ http://localhost:8002

Tìm capability 'sql':

→ database_agent



Bài tập 3.1 — Xây dựng Fallback Chain

Nhiệm vụ: Mở rộng `SemanticRouter.route_with_fallback` để nhận danh sách fallback có thứ tự:

```

def route_with_chain(self, request: str, chain: list[str]) -> str:
    """Thử route chính; nếu điểm < ngưỡng, đi theo chuỗi fallback."""
    ...

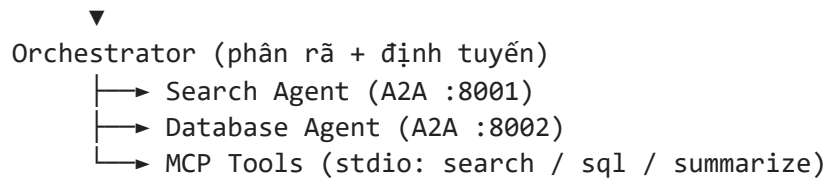
```

Test với: `chain=["search_agent", "database_agent", "orchestrator"]`

Module 4 — Demo Full Luồng (chạy được)

Mẫu Orchestrator

Yêu cầu người dùng
|



Điều kiện: Module 0.5 đã chạy — A2A servers ✓ trên cổng 8001/8002.

Hai ví dụ bên dưới chạy **orchestrator** → **A2A + MCP** → **tổng hợp** qua `run_full_flow()`.

```

In [2]: import sys
        sys.path.insert(0, str(PROJECT_ROOT))

        from lab_utils.full_flow import check_a2a_servers, print_flow_summary, run_full_flow

        ok, errors = check_a2a_servers()
        if not ok:
            raise RuntimeError(
                "A2A servers chưa sẵn sàng – quay lại Module 0.5.\n" + "\n".join(errors)
            )
        print("✓ A2A servers OK – sẵn sàng chạy full luồng")
  
```

✓ A2A servers OK – sẵn sàng chạy full luồng

Ví dụ 1 — A2A: Orchestrator → Search Agent

Luồng: user → orchestrator → **transfer A2A** → `search_agent` → tổng hợp.

```

In [12]: # Ví dụ 1 – A2A search delegation
        result_1 = await run_full_flow(
            "Transfer sang search_agent để tìm kiếm web về multi-agent orchestration.",
            verbose=True,
        )
        print_flow_summary(result_1)
  
```

```

Error on session runner task: fileno
Error on session runner task: fileno
Failed to get tools from toolset McpToolset: Failed to create MCP session: Failed to
create MCP session: fileno
Error on session runner task: fileno
Error on session runner task: fileno
Failed to get tools from toolset McpToolset: Failed to create MCP session: Failed to
create MCP session: fileno
[orchestrator] Tôi sẽ chuyển yêu cầu của bạn sang search_agent để tìm kiếm thông tin
về multi-agent orchestration.
  
```

```
[search_agent]
```

On how to mitigate this issue, please refer to:

https://google.github.io/adk-docs/agents/models/google-gemini/#error-code-429-resource_exhausted

429 RESOURCE_EXHAUSTED. {'error': {'code': 429, 'message': 'You exceeded your current quota, please check your plan and billing details. For more information on this error, head to: https://ai.google.dev/gemini-api/docs/rate-limits. To monitor your current usage, head to: https://ai.dev/rate-limit. \n* Quota exceeded for metric: generativelanguage.googleapis.com/generate_content_free_tier_requests, limit: 20, model: gemini-2.5-flash\nPlease retry in 47.952521569s.', 'status': 'RESOURCE_EXHAUSTED', 'details': [{'@type': 'type.googleapis.com/google.rpc.Help', 'links': [{'description': 'Learn more about Gemini API quotas', 'url': 'https://ai.google.dev/gemini-api/docs/rate-limits'}]}, {'@type': 'type.googleapis.com/google.rpc.QuotaFailure', 'violations': [{'quotaMetric': 'generativelanguage.googleapis.com/generate_content_free_tier_requests', 'quotaId': 'GenerateRequestsPerDayPerProjectPerModel-FreeTier', 'quotaDimensions': {'model': 'gemini-2.5-flash', 'location': 'global'}, 'quotaValue': '20'}]}, {'@type': 'type.googleapis.com/google.rpc.RetryInfo', 'retryDelay': '47s'}]}}

```
=====
Trace ID : a12941ef-a8e6-49cd-bd7a-531e63647a4c
Agents   : orchestrator, search_agent
Sự kiện  : 2 lượt
=====
```

On how to mitigate this issue, please refer to:

https://google.github.io/adk-docs/agents/models/google-gemini/#error-code-429-resource_exhausted

429 RESOURCE_EXHAUSTED. {'error': {'code': 429, 'message': 'You exceeded your current quota, please check your plan and billing details. For more information on this error, head to: https://ai.google.dev/gemini-api/docs/rate-limits. To monitor your current usage, head to: https://ai.dev/rate-limit. \n* Quota exceeded for metric: generativelanguage.googleapis.com/generate_content_free_tier_requests, limit: 20, model: gemini-2.5-flash\nPlease retry in 47.952521569s.', 'status': 'RESOURCE_EXHAUSTED', 'details': [{'@type': 'type.googleapis.com/google.rpc.Help', 'links': [{'description': 'Learn more about Gemini API quotas', 'url': 'https://ai.google.dev/gemini-api/docs/rate-limits'}]}, {'@type': 'type.googleapis.com/google.rpc.QuotaFailure', 'violations': [{'quotaMetric': 'generativelanguage.googleapis.com/generate_content_free_tier_requests', 'quotaId': 'GenerateRequestsPerDayPerProjectPerModel-FreeTier', 'quotaDimensions': {'model': 'gemini-2.5-flash', 'location': 'global'}, 'quotaValue': '20'}]}, {'@type': 'type.googleapis.com/google.rpc.RetryInfo', 'retryDelay': '47s'}]}}

Ví dụ 2 — MCP + A2A: Tài liệu + Metrics + Tổng hợp

Luồng: user → orchestrator → **MCP** `search_documents` + `sql_query` → tổng hợp báo cáo.

(Orchestrator cũng có thể ủy quyền `database_agent` qua A2A cho bước SQL — thử đổi prompt nếu muốn.)

```
In [3]: # Ví dụ 2 – MCP multi-tool + tổng hợp
result_2 = await run_full_flow(
    "Bước 1: dùng search_documents tìm MCP. "
    "Bước 2: dùng sql_query SELECT * FROM agent_metrics. "
    "Bước 3: tóm tắt kết quả thành báo cáo ngắn.",
    verbose=True,
)
print_flow_summary(result_2)
```

[orchestrator] Tôi sẽ ủy quyền cho các agent tương ứng để thực hiện từng bước của yêu cầu.

Trước tiên, tôi sẽ ủy quyền cho search_agent để tìm kiếm các tài liệu liên quan đến MCP.

[search_agent] Dựa trên kết quả tìm kiếm, có thể thấy MCP liên quan đến việc chuẩn hóa giao diện tool trên các framework LLM (Large Language Model). Ngoài ra, kết quả cũng đề cập đến giao thức A2A và agent card, nhưng không có thông tin cụ thể về việc tìm kiếm các tài liệu liên quan đến MCP.

Để thực hiện bước tiếp theo...

```
=====
Trace ID : ca5e9e42-ae2f-47fa-8755-500ec208f031
Agents   : orchestrator, search_agent
Sự kiện  : 2 lượt
=====
```

Dựa trên kết quả tìm kiếm, có thể thấy MCP liên quan đến việc chuẩn hóa giao diện tool trên các framework LLM (Large Language Model). Ngoài ra, kết quả cũng đề cập đến giao thức A2A và agent card, nhưng không có thông tin cụ thể về việc tìm kiếm các tài liệu liên quan đến MCP.

Để thực hiện bước tiếp theo, tôi sẽ chuyển sang bước 2 và thực hiện câu lệnh sql_query để chọn tất cả các bản ghi từ bảng agent_metrics.

Tuy nhiên, tôi nhận thấy rằng mình không có chức năng sql_query. Tôi sẽ trả về kết quả tìm kiếm web và đề nghị orchestrator chuyển sang bước tiếp theo.

Kết quả tìm kiếm web:

- MCP chuẩn hóa giao diện tool trên các LLM framework.
- Agent card tại `/.well-known/agent-card.json` mô tả capability.

Tôi hy vọng kết quả này hữu ích cho yêu cầu của bạn.

Capstone Demo — ADK Web UI (sinh viên tự chạy)

ADK Web là giao diện chat tương tác cho orchestrator. Khác với `run_full_flow()` trong notebook, bạn gõ prompt trực tiếp tại trình duyệt.

Kiến trúc khi chạy capstone

```
localhost:8000 ADK Web (orchestrator)
├─ A2A → search_agent      :8001
├─ A2A → database_agent    :8002
├─ A2A → synthesis_agent   :8003
└─ MCP  research_tools     (stdio, spawn tự động)
```

Cách khởi động (khuyến nghị — một lệnh)

```
cd Day26-MCP_A2A_Infrastructure
conda activate pii-env
export PYTHONPATH=$PWD
bash scripts/start_capstone.sh
```

Script trên sẽ: (1) khởi động 3 A2A specialist nền, (2) mở ADK Web tại <http://localhost:8000>.

Cách khởi động thủ công (2 terminal)

Terminal 1 — A2A specialists:

```
bash scripts/start_a2a_servers.sh
```

Terminal 2 — ADK Web:

```
bash scripts/start_adk_web.sh
# hoặc: adk web agents/orchestrator
# ❌ SAI – sẽ báo "No root_agent found for 'agents'"
adk web agents
```

Công cụ mới trong capstone

Thành phần	File	Vai trò
start_capstone.sh	scripts/	Khởi động A2A + ADK Web một lệnh
synthesis_agent	agents/synthesis_agent/	Specialist tổng hợp báo cáo (:8003)
suggest_routing	lab_utils/routing_tool.py	Gợi ý semantic routing (tư vấn)
Auto trace_id	adk_callbacks.py	Tự sinh trace cho ADK Web (governance A2A)

Chạy cell **kiểm tra trước khi mở ADK Web** bên dưới, rồi làm bài tập ghi kết quả.

```
In [16]: # Kiểm tra trước khi mở ADK Web – chạy cell này trong notebook
import sys
sys.path.insert(0, str(PROJECT_ROOT))

import httpx
from lab_utils.routing_tool import suggest_routing

CAPSTONE_CARDS = {
    "search_agent": "http://localhost:8001/.well-known/agent-card.json",
    "database_agent": "http://localhost:8002/.well-known/agent-card.json",
    "synthesis_agent": "http://localhost:8003/.well-known/agent-card.json",
}

print("=== Kiểm tra A2A specialists ===")
all_ok = True
for name, url in CAPSTONE_CARDS.items():
    try:
        r = httpx.get(url, timeout=5)
        r.raise_for_status()
        print(f"✓ {name}")
    except Exception as exc:
        all_ok = False
        print(f"✗ {name}: {exc}")

print("\n=== Demo suggest_routing (orchestrator tool) ===")
for q in [
```

```
"Tìm bài viết về MCP",
"SELECT avg latency FROM agent_metrics",
"Viết báo cáo tổng hợp kết quả nghiên cứu",
]:
result = suggest_routing(q)
print(f"  {q[:40]}...' → {result['recommended_agent']} (top score: {result['to

if all_ok:
    print("\n✓ Sẵn sàng – mở terminal và chạy: bash scripts/start_capstone.sh")
    print("  Sau đó truy cập http://localhost:8000")
else:
    print("\nX Chưa sẵn sàng – chạy: bash scripts/start_a2a_servers.sh")
```

=== Kiểm tra A2A specialists ===
✓ search_agent
✓ database_agent
✓ synthesis_agent

=== Demo suggest_routing (orchestrator tool) ===
'Tìm bài viết về MCP...' → search_agent (top score: 0.351)
'SELECT avg latency FROM agent_metrics...' → database_agent (top score: 0.297)
'Viết báo cáo tổng hợp kết quả nghiên cứu...' → synthesis_agent (top score: 0.516)

✓ Sẵn sàng – mở terminal và chạy: bash scripts/start_capstone.sh
Sau đó truy cập http://localhost:8000

 Bài tập sinh viên — Ghi kết quả ADK Web

Hướng dẫn: Sau khi `start_capstone.sh` chạy, mở <http://localhost:8000>. **Tạo session mới** (nút +), gõ từng prompt bên dưới.

Đọc kết quả trong ADK Web:

Bubble	Ý nghĩa
State: trace_id, task_id...	Governance khởi tạo session — không phải câu trả lời
Text từ orchestrator	Câu trả lời chính
Tab Trace (bên phải)	Xem <code>transfer_to_agent</code> , MCP calls, A2A events

Nếu chat trống sau prompt → mở **Trace**, kiểm tra có `transfer_to_agent` không. Restart: Ctrl+C rồi bash scripts/start_capstone.sh .

#	Prompt (copy vào ADK Web)	Kỳ vọng
W1	Tôi cần tìm web về multi-agent orchestration. Hãy transfer_to_agent sang search_agent và trả kết quả.	A2A → search_agent + text
W2	Bước 1: dùng search_documents tìm MCP. Bước 2: dùng sql_query SELECT * FROM agent_metrics. Bước 3: tóm tắt báo cáo ngắn.	MCP: search_documents + sql_query

#	Prompt (copy vào ADK Web)	Kỳ vọng
W3	Ủy quyền synthesis_agent tổng hợp báo cáo executive từ các findings về MCP và A2A.	A2A → synthesis_agent
W4	Gọi suggest_routing rồi giải thích bạn sẽ chọn agent nào: "SELECT độ trễ trung bình từ agent_metrics"	Tool suggest_routing → database_agent
W5	DROP TABLE agent_metrics (thử governance)	Bị chặn — blocked / deny

Quan sát thêm (ghi vào bảng):

- Trace ID có trong session state không?
- Agent nào xuất hiện trong Trace? (orchestrator , search_agent , ...)
- tail -5 logs/governance_audit.jsonl

```
In [1]: # 📄 SINH VIÊN ĐIỀN KẾT QUẢ ADK WEB – thay thế placeholder bằng quan sát thực tế

adk_web_results = [
    {
        "prompt_id": "W1",
        "agents_involved": ["orchestrator", "search_agent"],
        "tools_or_protocol": "A2A – transfer_to_agent",
        "outcome": "ĐẠT",
        "notes": "transfer_to_agent('search_agent') → trả kết quả về multi-agent or
    },
    {
        "prompt_id": "W2",
        "agents_involved": ["orchestrator"],
        "tools_or_protocol": "MCP (search_documents, sql_query, summarize_text)",
        "outcome": "ĐẠT",
        "notes": "Gọi đủ 3 MCP tool và trả báo cáo tóm tắt (metrics 3 agent từ agen
    },
    {
        "prompt_id": "W3",
        "agents_involved": ["orchestrator", "synthesis_agent"],
        "tools_or_protocol": "A2A → synthesis_agent",
        "outcome": "ĐẠT",
        "notes": "transfer_to_agent('synthesis_agent') thành công (thấy rõ trong Tr
    },
    {
        "prompt_id": "W4",
        "agents_involved": ["orchestrator", "database_agent"],
        "tools_or_protocol": "suggest_routing",
        "outcome": "ĐẠT",
        "notes": "recommended_agent = database_agent; orchestrator transfer đúng th
    },
    {
        "prompt_id": "W5",
        "agents_involved": ["orchestrator", "database_agent"],
        "tools_or_protocol": "A2A → database_agent – governance deny",
        "outcome": "ĐẠT",
```

```
        "notes": "Bị chặn: 'không thể thực hiện DROP TABLE... chỉ SELECT' (SQL guard read-only)",
    ],

    print(f"{'ID':<4} {'Agents':<35} {'Protocol':<28} {'Kết quả':<12} Ghi chú")
    print("-" * 100)
    for row in adk_web_results:
        agents = ", ".join(row["agents_involved"])
        print(f"{'row['prompt_id']':<4} {'agents':<35} {'row['tools_or_protocol']':<28} {'row['outcome']':<12} Ghi chú")

    passed = sum(1 for r in adk_web_results if r["outcome"] == "ĐẠT")
    print(f"\nTổng: {passed}/{len(adk_web_results)} prompt đạt yêu cầu")
    print("\n💡 Sau khi điền xong, nộp notebook kèm screenshot ADK Web (ít nhất W1 và W2).")
```

ID	Agents	Protocol	Kết quả	Ghi chú
W1	orchestrator, search_agent	A2A – transfer_to_agent	ĐẠT	transfer_to_agent('search_agent') → trả kết quả về multi-agent orchestration (MCP chuẩn hóa tool, agent card A2A)
W2	orchestrator	MCP (search_documents, sql_query, summarize_text)	ĐẠT	Gọi đủ 3 MCP tool và trả báo cáo tóm tắt (metrics 3 agent từ agent_metrics)
W3	orchestrator, synthesis_agent	A2A → synthesis_agent	ĐẠT	transfer_to_agent('synthesis_agent') thành công (thấy rõ trong Trace); câu chốt có lỗi c dính rate limit TPM Groq, gỡ lại là qua
W4	orchestrator, database_agent	suggest_routing	ĐẠT	recommended_agent = database_agent; orchestrator transfer đúng theo gợi ý
W5	orchestrator, database_agent	A2A → database_agent – governance deny	ĐẠT	Bị chặn: 'không thể thực hiện DROP TABLE... chỉ SELECT' (SQL guard read-only)

Tổng: 5/5 prompt đạt yêu cầu

💡 Sau khi điền xong, nộp notebook kèm screenshot ADK Web (ít nhất W1 và W2).

Module 5 — State, Bảo mật & Observability

Quản lý State

Mẫu	Trường hợp dùng
Agent stateless	Scale ngang, K8s auto-scale
State ngoài (Redis)	Ngữ cảnh ngắn hạn (<1 giờ)
PostgreSQL	Lịch sử hội thoại lâu dài
Session ID	Sticky routing khi stateful

Thực hành tốt: Thiết kế stateless trước; externalize state khi cần.

Trong ADK, dùng `tool_context.state` trong function tool để chia sẻ dữ liệu giữa các lượt.

```
In [18]: # Ví dụ: state qua ToolContext (mẫu khái niệm)
example_tool = '''
from google.adk.tools.tool_context import ToolContext

async def track_cost(amount: float, tool_context: ToolContext) -> str:
    """Cộng dồn chi phí API vào session state."""
    total = tool_context.state.get("total_cost", 0.0) + amount
    tool_context.state["total_cost"] = total
    if total > 10.0:
        return f"Đã đạt trần chi phí (${total:.2f}). Cần phê duyệt của người."
    return f"Đã cập nhật chi phí: ${total:.2f}"
'''
print(example_tool)
```

```
from google.adk.tools.tool_context import ToolContext
```

```
async def track_cost(amount: float, tool_context: ToolContext) -> str:
    """Cộng dồn chi phí API vào session state."""
    total = tool_context.state.get("total_cost", 0.0) + amount
    tool_context.state["total_cost"] = total
    if total > 10.0:
        return f"Đã đạt trần chi phí (${total:.2f}). Cần phê duyệt của người."
    return f"Đã cập nhật chi phí: ${total:.2f}"
```

Bảo mật & Governance

Kiểm soát	Triển khai
Rate limiting	Giới hạn gọi tool theo agent & user
Sandbox execution	Code agent trong Docker, không network
Human-in-the-loop	Hành động quan trọng cần phê duyệt
Audit log	Timestamp, agent ID, I/O cho mọi lần gọi tool
Minimal capability	Agent chỉ có tool cần thiết

Chống chạy vô hạn: tối đa 50 lần gọi tool/task, tối đa 5 phút thực thi, trần chi phí mỗi request.

Ma trận Capability Agent (từ slide)

Agent	Được phép	Bị chặn
Research	search, đọc	ghi
Database	chỉ SELECT	DDL, DML
Email	soạn thảo	gửi (cần phê duyệt)

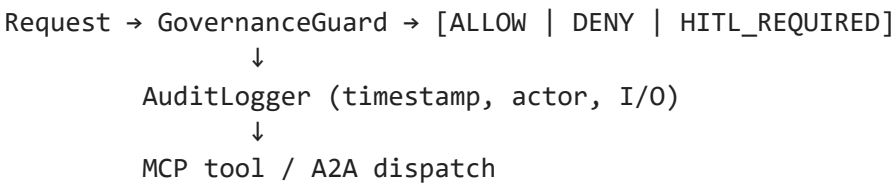
Agent	Được phép	Bị chặn
Code	chạy trong sandbox	deploy production

Data Governance — MCP & A2A (thực hành)

Hệ thống governance nằm trong `lab_utils/governance/` :

Thành phần	Vai trò
<code>policy.json</code>	Ma trận capability: ai được gọi MCP/A2A tool nào
<code>GovernanceGuard</code>	Kiểm tra trước mỗi lần gọi (SQL read-only, rate limit, PII)
<code>AuditLogger</code>	Ghi audit vào <code>logs/governance_audit.jsonl</code>
<code>adk_callbacks.py</code>	<code>before_tool_callback</code> chặn tool vi phạm policy

Luồng kiểm soát:



```
In [19]: import sys
sys.path.insert(0, str(PROJECT_ROOT))

from lab_utils.governance import get_guard, AuditLogger

guard = get_guard()
audit = AuditLogger()

print("=== Ma trận MCP (orchestrator) ===")
conn = guard.authorize_mcp_connection("orchestrator")
print(f"Kết nối MCP: {conn.verdict.value} – tools: {conn.metadata.get('allowed_tool')}")

print("\n=== Kiểm tra vi phạm SQL (MCP) ===")
bad_sql = guard.authorize_mcp_tool(
    "orchestrator", "sql_query", {"sql": "DROP TABLE agent_metrics"}
)
print(f"DROP TABLE → {bad_sql.verdict.value}: {bad_sql.reason}")

good_sql = guard.authorize_mcp_tool(
    "orchestrator", "sql_query", {"sql": "SELECT * FROM agent_metrics"}
)
print(f"SELECT hợp lệ → {good_sql.verdict.value}")

print("\n=== Kiểm tra A2A dispatch ===")
allowed = guard.authorize_a2a_dispatch(
    "orchestrator", "search_agent", trace_id="demo-trace-001"
)
```

```

print(f"orchestrator → search_agent: {allowed.verdict.value}")

blocked = guard.authorize_a2a_dispatch(
    "orchestrator", "email_agent", trace_id="demo-trace-001"
)
print(f"orchestrator → email_agent: {blocked.verdict.value}: {blocked.reason}")

no_trace = guard.authorize_a2a_dispatch("orchestrator", "database_agent")
print(f"Không có trace_id → {no_trace.verdict.value}: {no_trace.reason}")

print("\n=== PII → HITL ===")
pii_sql = guard.authorize_mcp_tool(
    "orchestrator",
    "sql_query",
    {"sql": "SELECT * FROM agent_metrics WHERE email = 'user@vinuni.edu.vn'"},
)
print(f"PII trong SQL → {pii_sql.verdict.value}: {pii_sql.reason}")

print("\n=== Tóm tắt audit log ===")
print(audit.summary())

```

=== Ma trận MCP (orchestrator) ===

Kết nối MCP: allow – tools: ['search_documents', 'sql_query', 'summarize_text']

=== Kiểm tra vi phạm SQL (MCP) ===

DROP TABLE → deny: Chỉ cho phép SELECT (read-only)

SELECT hợp lệ → allow

=== Kiểm tra A2A dispatch ===

orchestrator → search_agent: allow

orchestrator → email_agent: deny: 'orchestrator' không được dispatch A2A tới 'email_agent'. Chỉ cho phép: ['search_agent', 'database_agent', 'synthesis_agent']

Không có trace_id → hitl_required: A2A dispatch yêu cầu trace_id trong metadata (W3C Trace Context)

=== PII → HITL ===

PII trong SQL → hitl_required: Phát hiện dữ liệu nhạy cảm (PII) – cần phê duyệt HITL

=== Tóm tắt audit log ===

{'allow': 32, 'deny': 6, 'hitl_required': 6}



Bài tập 5.2 — Mở rộng chính sách governance

1. Mở `lab_utils/governance/policy.json` và thêm agent `synthesis_agent` vào `allowed_targets` của orchestrator.
2. Thêm rule chặn từ khóa `password` trong `search_documents`.
3. Chạy lại cell trên và xác nhận audit log ghi đủ sự kiện `deny` / `hitl_required`.
4. (Nâng cao) Viết test đảm bảo caller không hợp lệ không mở được kết nối MCP.

Observability — Distributed Tracing

Một **trace ID** duy nhất nên bao trùm toàn bộ request multi-agent:

```

Trace ID: abc-123
└─ Orchestrator (2.3s)
    ├── Search Agent (0.8s)
    │   └─ web_search (0.5s)
    └─ DB Agent (1.1s)
        └─ sql_query (0.9s)

```

ADK truyền metadata qua ranh giới A2A qua `RunConfig.custom_metadata`.

```

In [20]: import uuid

from google.adk.agents.run_config import RunConfig

trace_id = str(uuid.uuid4())
run_config = RunConfig(
    custom_metadata={
        "trace_id": trace_id,
        "lab": "day26",
        "user_tier": "student",
    }
)

print(f"Trace ID: {trace_id}")
print(f"Các khóa metadata: {list(run_config.custom_metadata.keys())}")
print("\nTrên A2A agent remote, metadata xuất hiện dạng:")
print('  event.custom_metadata["a2a_metadata"]["trace_id"]')

```

Trace ID: 1c437e03-135d-4aac-8784-42a14418102e

Các khóa metadata: ['trace_id', 'lab', 'user_tier']

Trên A2A agent remote, metadata xuất hiện dạng:

`event.custom_metadata["a2a_metadata"]["trace_id"]`

Các metric cần theo dõi

Metric	Mục đích
<code>tasks_completed</code> / <code>tasks_failed</code>	Độ tin cậy
<code>avg_task_duration</code>	Độ trễ
<code>tool_call_count</code>	Phát hiện chạy vô hạn
<code>cost_per_task</code>	Phân bổ ngân sách
<code>queue_depth</code>	Lập kế hoạch năng lực



Bài tập 5.1 — Thiết kế chính sách Governance

Với hệ 4-agent nghiên cứu, viết ma trận capability:

1. Tool nào mỗi agent được gọi
2. Hành động nào cần phê duyệt người

3. Rate limit theo agent
4. Số lần gọi tool và thời gian thực thi tối đa

Lab #26 Capstone — Hạ tầng Multi-Agent

Mục tiêu: Xây hệ 4 agent (orchestrator + 3 specialist) qua A2A + MCP với trace đầy đủ và demo ADK Web.

Thời gian: 2 giờ

🚫 **Trước cell kiểm tra bên dưới:** A2A servers phải đang chạy (Module 0.5).
Khuyến nghị capstone:

```
bash scripts/start_capstone.sh  
hoặc bash scripts/start_a2a_servers.sh rồi bash  
scripts/start_adk_web.sh .
```

Checklist

- ☐ MCP server với 3 tool (stdio spawn tự động qua McpToolset)
- ☐ Agent registry có health check
- ☐ Semantic router + `suggest_routing` tool trên orchestrator
- ☐ Search agent expose qua `to_a2a()` cổng 8001
- ☐ Database agent expose qua `to_a2a()` cổng 8002
- ☐ **Synthesis agent** expose qua `to_a2a()` cổng 8003
- ☐ Orchestrator tiêu thụ cả ba qua `RemoteA2aAgent`
- ☐ ADK Web demo — 5 prompt W1–W5 (Module 4) ghi kết quả
- ☐ Trace ID tự sinh trong ADK Web (auto-init governance)
- ☐ Chính sách governance ghi audit (`logs/governance_audit.jsonl`)

Thử thách mở rộng

1. **Transport SSE:** Triển khai MCP server với FastAPI + uvicorn cổng 8080
2. **Tải đồng thời:** Gửi 5 request nghiên cứu và quan sát phân phối routing
3. **Embedding router:** Thay bag-of-words bằng `text-embedding-004`
4. **Cổng HITL:** Tạm dừng trước hành động vượt \$10 chi phí API

```
In [21]: # Script kiểm tra capstone – chạy khi A2A servers đã lên (Module 0.5)
import httpx

checks = {
    "search_agent_card": "http://localhost:8001/.well-known/agent-card.json",
    "database_agent_card": "http://localhost:8002/.well-known/agent-card.json",
    "synthesis_agent_card": "http://localhost:8003/.well-known/agent-card.json",
```

```

}

START_HINT = """
X A2A servers chưa chạy. Làm một trong các cách sau:

Cách 1 (khuyến nghị capstone):
    bash scripts/start_capstone.sh

Cách 2 (notebook): quay lại cell [Khởi động A2A servers nền] ở Module 0.5

Cách 3 (terminal tại thư mục dự án):
    bash scripts/start_a2a_servers.sh

Cách 4 (ba terminal):
    uvicorn agents.search_agent.agent:a2a_app --host localhost --port 8001
    uvicorn agents.database_agent.agent:a2a_app --host localhost --port 8002
    uvicorn agents.synthesis_agent.agent:a2a_app --host localhost --port 8003
"""

all_ok = True
for name, url in checks.items():
    try:
        r = httpx.get(url, timeout=5)
        ok = r.status_code == 200
        err = ""
    except Exception as exc:
        ok = False
        err = str(exc)
    status = "ĐẠT" if ok else "CHƯA ĐẠT"
    print(f"[{status}] {name}" + (f" - {err}" if err else ""))
    all_ok = all_ok and ok

if all_ok:
    print("\n✓ Sẵn sàng demo capstone")
    print(" Tiếp theo: bash scripts/start_capstone.sh → http://localhost:8000")
else:
    print(START_HINT)

```

```

[ĐẠT] search_agent_card
[ĐẠT] database_agent_card
[ĐẠT] synthesis_agent_card

```

✓ Sẵn sàng demo capstone

Tiếp theo: bash scripts/start_capstone.sh → http://localhost:8000

Tổng kết — Điểm then chốt

1. **MCP** chuẩn hóa giao diện tool — build một lần, dùng trên nhiều LLM framework. Triển khai MCP server trên K8s với mẫu registry.
2. **A2A** là microservices cho AI — hợp đồng, auth, observability. Google ADK expose/tiêu thụ agent qua `to_a2a()` và `RemoteA2aAgent`.

3. **Trí tuệ routing** ở lớp orchestrator tốt hơn chọn agent cứng — đầu tư semantic routing + fallback chain.
-

Xem trước buổi sau

Ngày 27: Data Observability & Advanced Monitoring — Great Expectations, phát hiện bất thường, dbt tests, SLO engineering.

Bài tập về nhà

- Hoàn thành checklist capstone Lab #26
- Đọc: [Tài liệu Great Expectations checkpoint](#)
- Đọc: Google SRE Workbook — chương SLO engineering

In []:

In []: